

Python Assignment 5

Question 1:

1.What are bytes in Python?
Why are they immutable?

Answer :

The bytes data type represents an **immutable sequence of bytes (8-bit values)**. Each element is an integer between **0 and 255**. It is mainly used to store and process binary data such as files, images, audio, network packets, and encrypted information.

Bytes are immutable to ensure **data integrity**, **better performance**, **hashability**, and **security**. Once a bytes object is created, its contents cannot be modified, making it suitable for safely handling binary data.

Example :

```
data = bytes([65, 66, 67])    #Storage format
print(data)    #b'ABC'

#ASCII Values
#65 → A
#66 → B
#67 → C

#converting the character
print(chr(data[0]))    #A
print(chr(data[1]))    #B
print(chr(data[2]))    #C
#data[0] = 69    #Error
# Once a bytes object is created, its contents cannot be modified.

#Iterating over Bytes
data = b"ABC"
for value in data:
    print(value)
#65
#66
#67
```

```
data = bytes([65,66,67])
```

Memory Representation

Index	0	1	2
Bytes	65	66	67

ASCII
65 → A
66 → B
67 → C

Python displays:
b'ABC'

Question 2:

Predict the output:

```
b = bytes([65, 66, 67])
```

```
print(b)
```

Explain how numbers are converted internally.

Answer :

```
b = bytes([65, 66, 67])      #Storage format
print(b)                     #b'ABC'

#ASCII Values
#65 → A
#66 → B
#67 → C
```

Question 3 :

What is the difference between bytes and bytearray?

Mention mutability and use cases.

Answer :

Both **bytes** and **bytearray** are binary data types in Python. They store a sequence of bytes (8-bit values), where each value ranges from **0 to 255**.

The **main difference** is **mutability**:

- **bytes** → Immutable (cannot be modified after creation)
- **bytearray** → Mutable (can be modified after creation)

```
data1 = bytes([65, 66, 67])
data2 = bytearray([65, 66, 67])

print(data1)
print(data2)

print(type(data1))
print(type(data2))

#try to Modify
data1[0] = 68    #Error
data2[0] = 68
```

```
#bytes → Immutable (cannot be modified after creation)
#bytearray → Mutable (can be modified after creation)
```

Question 4:

Predict the output:

```
ba = bytearray([65, 66, 67])
```

```
ba[0] = 97
```

```
print(ba)
```

Why is this allowed?

Answer :

Yes Allowed, #bytearray → Mutable (can be modified after creation)

```
ba = bytearray([65, 66, 67])
```

```
ba[0] = 97
```

```
print(ba)
```

```
#bytearray(b'aBC')
```

```
#ASCII
```

```
65 - A
```

```
66 - B
```

```
67 - C
```

```
97 - a
```

Questions 5:

5. What is None in Python?

Is it the same as 0, False, or empty string?

Answer :

No, 0 Consider as False value in boolean and none is consider as absent value but memory allocated.

None is a special built-in constant in Python that represents the **absence of a value** or **no value**. It is the **only object** of the NoneType data type.

```
result = None
```

```
print(result)
```

```
print(type(result))
```

OUTPUT :

```
None
```

```
<class 'NoneType'>
```

Value	Data Type	Meaning
None	NoneType	No value / Absence of value

Value	Data Type	Meaning
0	int	Integer zero
False	bool	Boolean false
""	str	Empty string
[]	list	Empty list

Question 6 :

Predict the output:

```
x = None
```

```
print(type(x))
```

```
print(x == False)
```

Explain the result.

Answer :

```
x = None
print(type(x))      #<class 'NoneType'>
print(x == False)   #False

# None ≠ 0
# None ≠ False
# None ≠ ""
```

Question 7:

What is the Boolean data type in Python?

List all values that evaluate to False.

Answer :

The **Boolean (bool)** data type represents **logical truth values**.

Stores only True or False

Immutable

Used in conditions and decision making

Result of comparison and logical operations

Internally represented as integers (True = 1, False = 0)

```
status = False

print(status)      #False
print(type(status)) #<class 'bool'>

print(10 > 5)      #True
print(10 < 5)      #False
print(10 == 10)    #True
print(20 != 20)    #False
```

```
print(True and False)    #False
print(True or False)     #True
print(not True)          #False
```

Question 8:

What is a set in Python?

Question :

A **set** is an **unordered, mutable** collection of **unique elements**.

It is used to store multiple values where **duplicate values are not allowed**.

Python automatically removes duplicate elements from a set.

```
class_A = {"Amar", "Akshay", "Rahul"}
class_B = {"Rahul", "Pooja", "Amar"}

print(class_A & class_B)
```

Question 9:

Predict the output:

s = {10, 20, 30, 10, 20}

print(s)

Why are some values missing?

Answer :

duplicate values are not allowed in Set data type so it avoid repeated value.

```
s = {10, 20, 30, 10, 20}
print(s)    #{10, 20, 30}
```

Question 10:

What is dynamic typing in Python?

Answer :

Dynamic Typing is a feature of Python in which the interpreter automatically determines the data type of a variable at runtime based on the value assigned to it. Therefore, explicit variable declaration is not required.

```
x = 10
print(type(x))

x = 3.14
print(type(x))

x = "Python"
print(type(x))
```

OUTPUT :

```
<class 'int'>  
<class 'float'>  
<class 'str'>
```